

Лабораторна робота №2

Проектування архітектури ПЗ та моделювання даних – DnD Companion

1. Технологічний стек та обґрунтування

Рівень	Технологія	Обґрунтування
Back-end	Node.js + Express.js	Відмінна підтримка WebSocket (Socket.io) для real-time синхронізації HP та стану сесії. Велика екосистема npm, швидка розробка REST API.
Front-end	React.js + TypeScript	Компонентна архітектура ідеально підходить для складних UI (аркуш персонажа, дашборд DM). TypeScript знижує кількість помилок у складних структурах даних (характеристики персонажа).
СКБД	PostgreSQL	Реляційна модель чудово підходить для складних зв'язків (персонаж – раса – клас – заклинання). Підтримує JSONB для гнучкого зберігання homebrew-контенту.
Real-time	Socket.io	Двостороння комунікація для синхронізації стану сесії між DM та гравцями.
DevOps	Docker + GitHub Actions	Контейнеризація гарантує однакове середовище розробки та продакшену. CI/CD через GitHub Actions автоматизує деплой.
Хостинг	Railway / Render	Безкоштовний tier для навчального проєкту, підтримує PostgreSQL та Node.js з коробки.

Обґрунтування вибору PostgreSQL: Дані в DnD-системі мають чітку реляційну структуру – персонаж належить до раси, має клас, знає заклинання, носить предмети. Всі ці зв'язки добре моделюються через FK та JOIN. При цьому homebrew-контент із непередбачуваною структурою зберігається у JSONB-полях, що дає гнучкість NoSQL без відмови від транзакцій та цілісності даних.

Обґрунтування React + Node.js: Спільна мова (JavaScript/TypeScript) на фронті та беку спрощує розробку, дозволяє ділитися типами та валідаційною логікою. Socket.io нативно інтегрується з Node.js і React, що критично для real-time оновлень під час ігрової сесії.

2. Архітектура C4

Рівень 1 – System Context Diagram

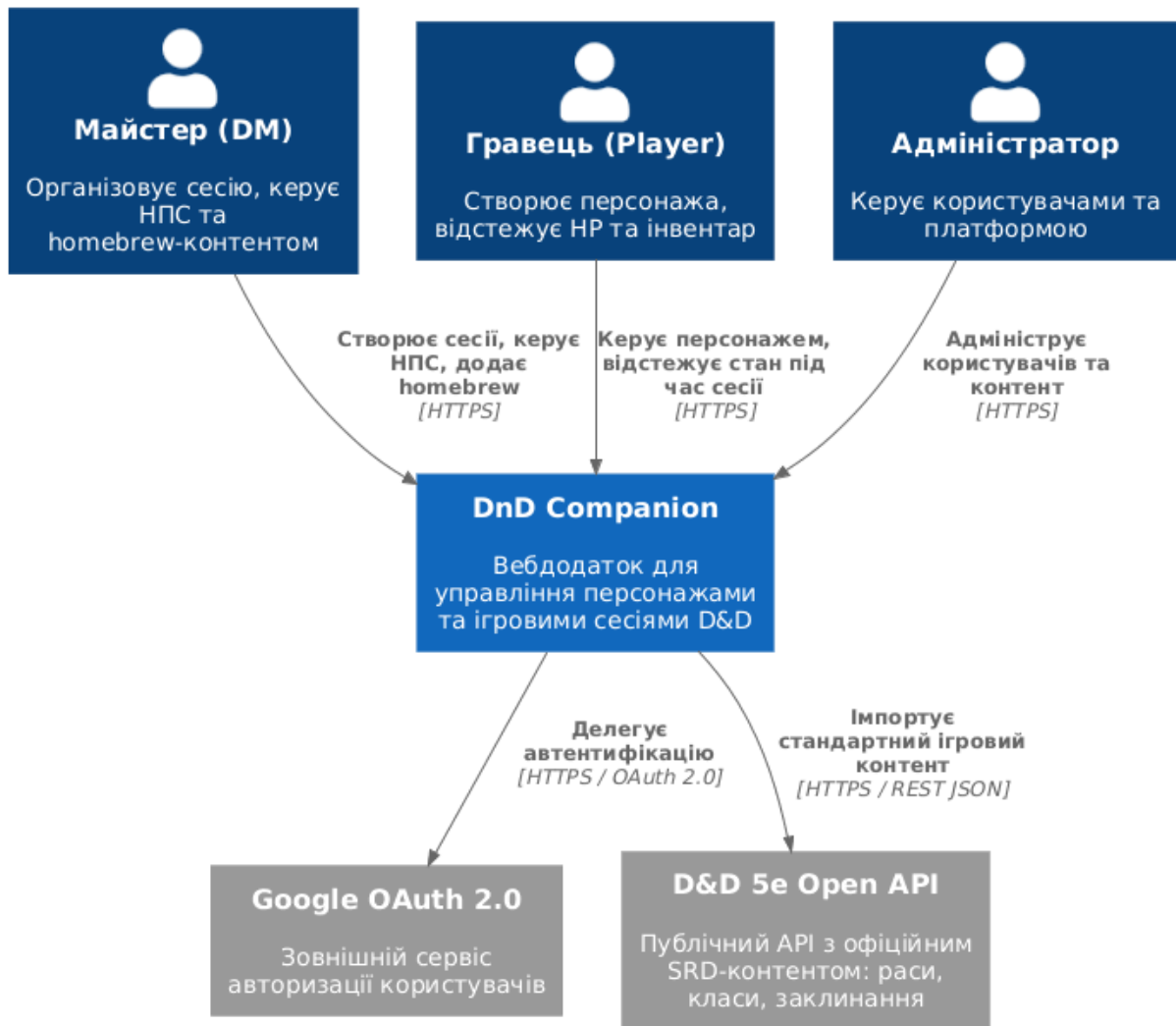


Рисунок 1 – Діаграма контексту системи (C4 Level 1: System Context)

Система взаємодіє з трьома типами користувачів. Авторизація делегується Google OAuth. Стандартний контент (раси, класи, заклинання з SRD) підтягується з публічного D&D 5e Open API при первинному наповненні бази даних.

Рівень 2 – Container Diagram

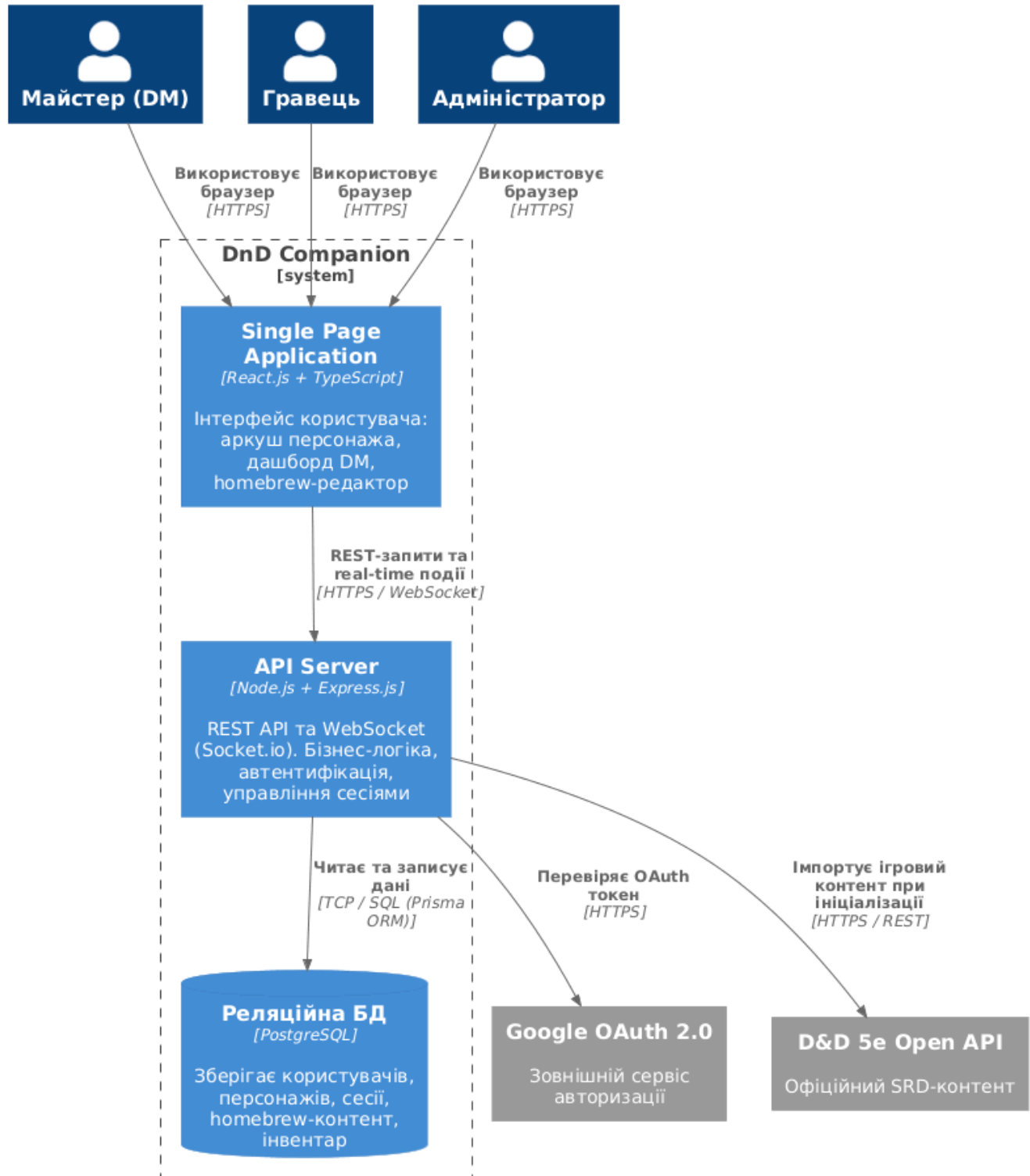


Рисунок 2 – Діаграма контейнерів системи (C4 Level 2: Container Diagram)

SPA у браузері комунікує з API-сервером через HTTPS (REST) та WebSocket (Socket.io) для real-time оновлень. API-сервер взаємодіє з PostgreSQL через TCP.

Авторизація через зовнішній Google OAuth, стандартний контент імпортується з D&D 5e Open API.

Рівень 3 – Component Diagram (API Server)

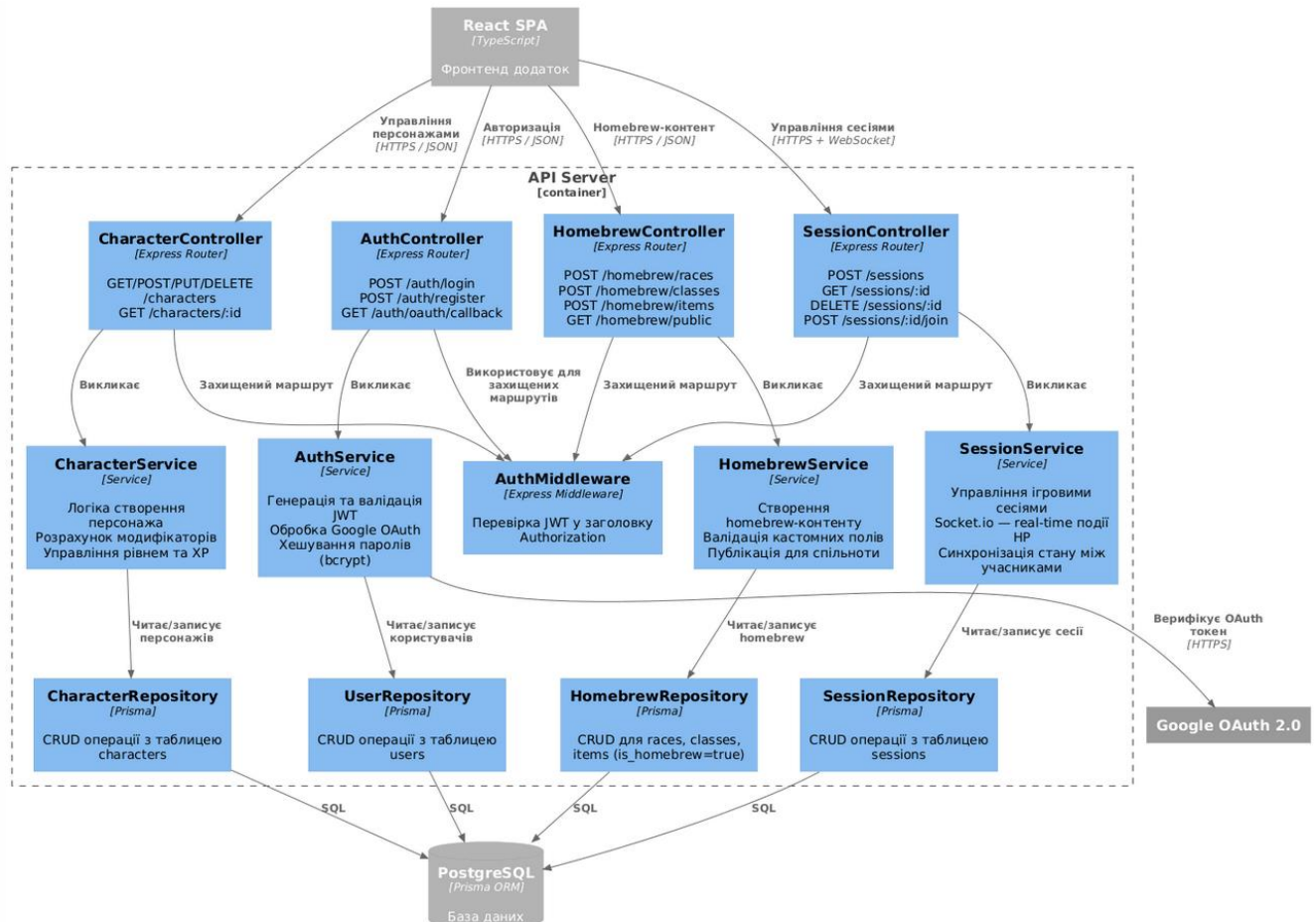


Рисунок 3 – Діаграма компонентів API-сервера (C4 Level 3: Component Diagram)

API побудований за трирівневою архітектурою Controller, Service, Repository. Controllers обробляють HTTP-запити, Services містять бізнес-логіку, Repositories відповідають за роботу з БД через ORM Prisma. Socket.io-логіка реального часу інкапсульована в SessionService.

3. Схеми бази даних (ER-модель, PostgreSQL)

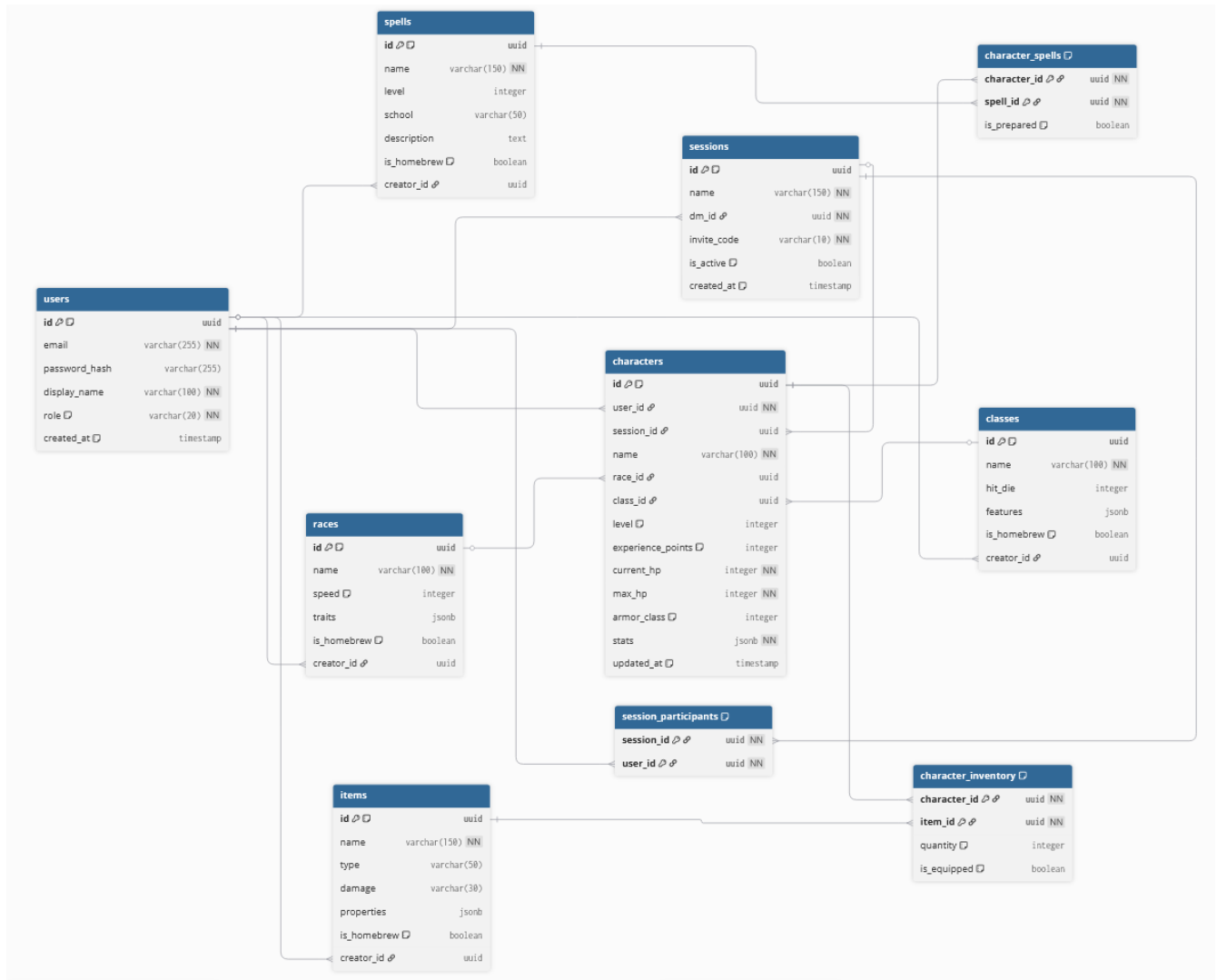


Рисунок 4 – ER-діаграма бази даних

Ключові рішення щодо схеми:

- **JSONB-поля** (stats, traits, features, properties) – використані для зберігання гнучкого homebrew-контенту, структура якого може відрізнятися для кожного запису. Це дозволяє уникнути надмірної кількості nullable-колонок.
- **Зв'язки M:N** через проміжні таблиці character_spells та character_inventory – персонаж може знати багато заклинань, одне заклинання може бути у багатьох персонажів.
- **Поле is_homebrew + creator_id** в таблицях races, classes, spells, items – єдиний підхід для розрізнення офіційного та кастомного контенту без дублювання таблиць.

- **Нормалізація до 3НФ** – раси та класи винесені в окремі таблиці, немає транзитивних залежностей.

Відповіді на контрольні запитання

1. Що таке модель C4 і для чого потрібен кожен з її рівнів (Context, Container, Component)?

C4 – ієрархічна модель для документування архітектури ПЗ на чотирьох рівнях деталізації. **Context** показує систему в оточенні (хто нею користується та з чим вона взаємодіє). **Container** розкриває основні виконувані блоки (застосунки, БД). **Component** деталізує внутрішню структуру одного контейнера (контролери, сервіси). **Code** (4-й рівень) – діаграми класів (зазвичай генеруються автоматично).

2. У чому принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних? Коли варто обирати кожную з них?

SQL (реляційні) – чітка схема, ACID-транзакції, потужні JOIN-запити. Підходять коли дані структуровані та між ними є складні зв'язки. NoSQL – гнучка схема, горизонтальне масштабування, зберігання документів/графів/пар ключ-значення. Підходять для великих обсягів неструктурованих або слабо пов'язаних даних (стрічки новин, логи, каталоги товарів).

3. Що таке теорема CAP (Consistency, Availability, Partition tolerance) і як вона впливає на вибір бази даних?

Розподілена система може одночасно гарантувати лише дві з трьох властивостей: **Consistency** (усі вузли бачать однакові дані), **Availability** (система завжди відповідає), **Partition Tolerance** (система працює при розриві мережі між

вузлами). Наприклад, PostgreSQL – СР-система (жертвує доступністю заради консистентності), Cassandra – АР-система.

4. Які переваги та недоліки мікросервісної архітектури порівняно з монолітною (на рівні Container diagram)?

Моноліт простіший у розробці, тестуванні та дебагінгу на початковому етапі, але важко масштабується і має спільну точку відмови. Мікросервіси дозволяють масштабувати окремі компоненти, деплоїти незалежно та використовувати різні технології для різних сервісів – але додають складність у комунікацію між сервісами, моніторинг та управління. Для навчального проєкту типу DnD Companion обґрунтованим є **моноліт** на початку з можливістю виділення сервісу real-time (Socket.io) в окремий контейнер у майбутньому.